

Cache

The problem with memory access

A computer has a problem with memory access. CPUs have always been faster than memories.

Consider a modern 4 GHz CPU.

- Each clock cycle takes **0.25** nanoseconds.
- DDR5 memory access needs about **80** nanoseconds.

In the time the CPU waits for **one** memory read, it could have *executed 320 instructions*.

Component	Typical latency
CPU cycle (4 GHz)	0.25 ns
L1 cache hit	~1 ns
L2 cache hit	~4 ns
L3 cache hit	~10 ns
DRAM (DDR5)	~80 ns
SSD (NVMe)	~10,000 ns

A single DRAM access costs **320 CPU cycles** worth of waiting.

Simple solutions

There are several ways to deal with the speed gap.

Wait states The simplest approach

- Start a memory read and stall the CPU until the data arrives.
- The CPU does nothing for hundreds of cycles.

Out-of-order execution

- The CPU keeps a window of decoded instructions.
- While waiting for one memory read, it executes other instructions that are ready.

These techniques help.

However, they cannot erase the *gap* between CPU speed and memory speed. The true solution is caching.

The memory hierarchy

We know how to make **very fast** memory.

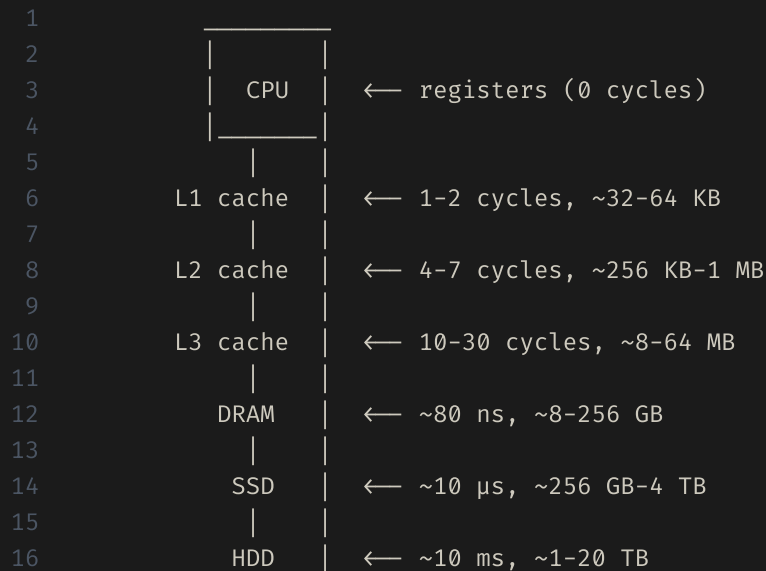
- *SRAM* (static RAM) uses six transistors per bit. It is **fast but expensive and dense**
- *DRAM* (dynamic RAM) uses one transistor and one capacitor per bit. It is **slow but cheap and dense**

The cost and speed of memory depend on *distance*.

Memory near the CPU must be small because it sits on the same die.

Memory far from the CPU can be large but takes time to reach.

PlanetScale: Caching



The memory hierarchy is a pyramid. Each level is *larger, slower* and **cheaper**.

What is caching

A cache is a small, fast memory that holds *copies of frequently used data* from a larger, slower memory.

The idea is *simple*.

1. Put the **most heavily** used memory words *into the cache*.
2. When the CPU needs a word, *look at the cache first*.
3. If the word is there, the CPU gets it in one or two cycles.
4. If the word is not there, the CPU must wait for the slow memory.

Caching works because most programs do not access memory randomly

Temporal and spatial locality

Caching works because of the **locality principle**.

Programs access data in *predictable* clusters.

There are two types of locality.

1. Temporal locality.

If a program uses a memory word *now*, it will probably use the same word again *soon*.

2. Spatial locality.

If a program uses a memory word, it will probably use *nearby words* next.

```
1  int sum_array(int* arr, int n) {  
2      int sum = 0;  
3      for (int i = 0; i < n; i++) {  
4          sum += arr[i];  
5      }  
6      return sum;  
7  }
```

- The variable `sum` shows *temporal locality*.
- The loop counter `i` shows *temporal locality*.
- The array access `arr[i]` shows *spatial locality*, because elements sit at **consecutive** memory addresses.

Hit ratio and mean access time

Formally, we have three symbols.

- c is the *cache access time*
- m is the *main memory access time*
- h is the *hit ratio*, the fraction of all references that the cache can satisfy

The hit ratio is often written as $h = (k - 1)/k$,

where k is the number of times a word is referenced and the miss ratio is $1 - h$.

The **mean access time** combines both cases:

$$\text{Mean access time} = c + (1 - h)m$$

When h approaches 1, almost all references come from the cache and the access time goes to c .

When h approaches 0, the CPU must go to main memory every time and the access time goes to $c + m$.

Example

Given:

- Cache hit time $c = 1$ ns
- DRAM miss time $m = 100$ ns
- Hit ratio $h = 0.95$

Computation:

$$\begin{aligned}t &= c + (1 - h)m \\&= 1 + (0.05)(100) \\&= 1 + 5 \\&= 6 \text{ ns}\end{aligned}$$

Hit ratio	Mean access time	Slowdown vs cache
0.90	11.0 ns	11.0x
0.95	6.0 ns	6.0x
0.98	3.0 ns	3.0x
0.99	2.0 ns	2.0x
0.999	1.1 ns	1.1x

A 99 percent hit ratio brings the system within 10 percent of pure cache speed.

Cache lines

With the *locality principle*, main memories and caches divide into fixed size blocks.

The blocks inside the cache are called **cache lines** (or cache blocks).

When a **cache miss** occurs,

- the computer loads the entire cache line from main memory.
- It does not load only the needed word.
- This decision exploits *spatial locality*.

Cache lines example

For example,

- with a **64**-byte line size,
- a reference to memory address **260** pulls a line that contains bytes **256** to **319** into one cache line.
- The next access to byte **300** is already a **hit**.